Listas de prioridades

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Introdução
- Implementação de listas de prioridades
- Alteração de prioridades
- Inserção e remoção
- Construção
- Ordenação
- Exercícios

Introdução

- Para algumas aplicações (ex. SO), prioridade associada aos dados é essencial
  - Operação realizada com frequência: encontrar dado com maior prioridade
    - Ex.: processo que irá executar na CPU
- Lista de prioridade: estrutura de dados onde dados estão relacionados a prioridades (valores numéricos)

Introdução

- Operações básicas em listas de prioridades:
  - Seleção do elemento de maior prioridade
  - Inserção de novo elemento
  - Remoção do elemento de maior prioridade
  - Alteração de prioridade

Implementação de listas de prioridades

- Implementação por lista não ordenada:
  - Seleção: O(n)
  - Inserção: O(1)
  - Remoção (inclui busca): O(n)
  - Alteração (inclui busca): O(n)
  - Construção da lista: O(n)
    - O(1) para cada elemento

Implementação de listas de prioridades

- Implementação por lista ordenada: elemento de maior prioridade é o primeiro
  - Seleção: O(1)
  - Inserção: O(n)
  - Remoção: O(1)
  - Alteração: O(n)
  - Construção: O(n log n), devido a ordenação

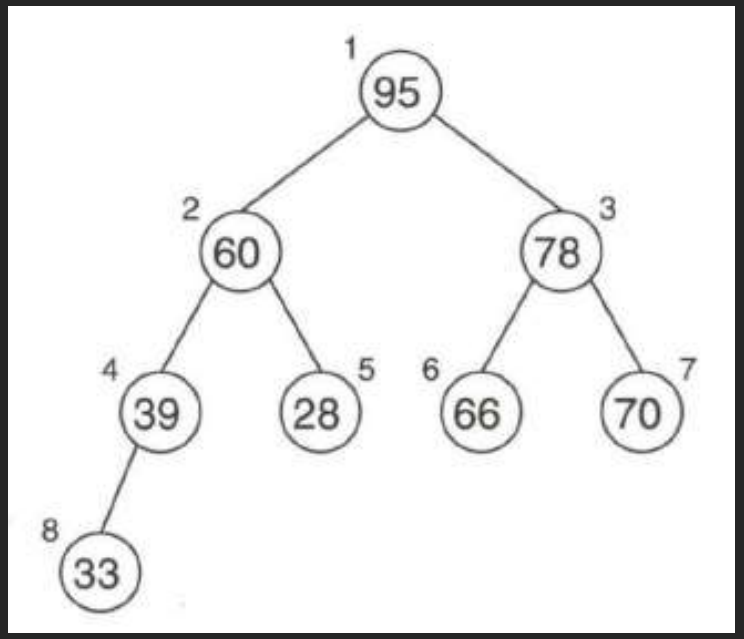
Implementação de listas de prioridades

- Implementação por heap: diminui complexidade da inserção, alteração e construção
  - Remoção é superior a O(1), mas é rápida
- Um heap é uma lista linear composta por elementos com chaves $s_1, \ldots, s_n$ onde é válida a seguinte propriedade: $s_i \leq s_{\lfloor i/2 \rfloor}$ para $1 < i \leq n$
  - Cada chave s_i corresponde à prioridade de um elemento
  - Árvore binária completa: facilita visualização da propriedade
    - Implementação não utiliza estrutura de dados do tipo árvore!

Implementação de listas de prioridades

- Exemplo de heap com 8 elementos e respectiva árvore binária completa:

95	60	78	39	28	66	70	33



Implementação de listas de prioridades

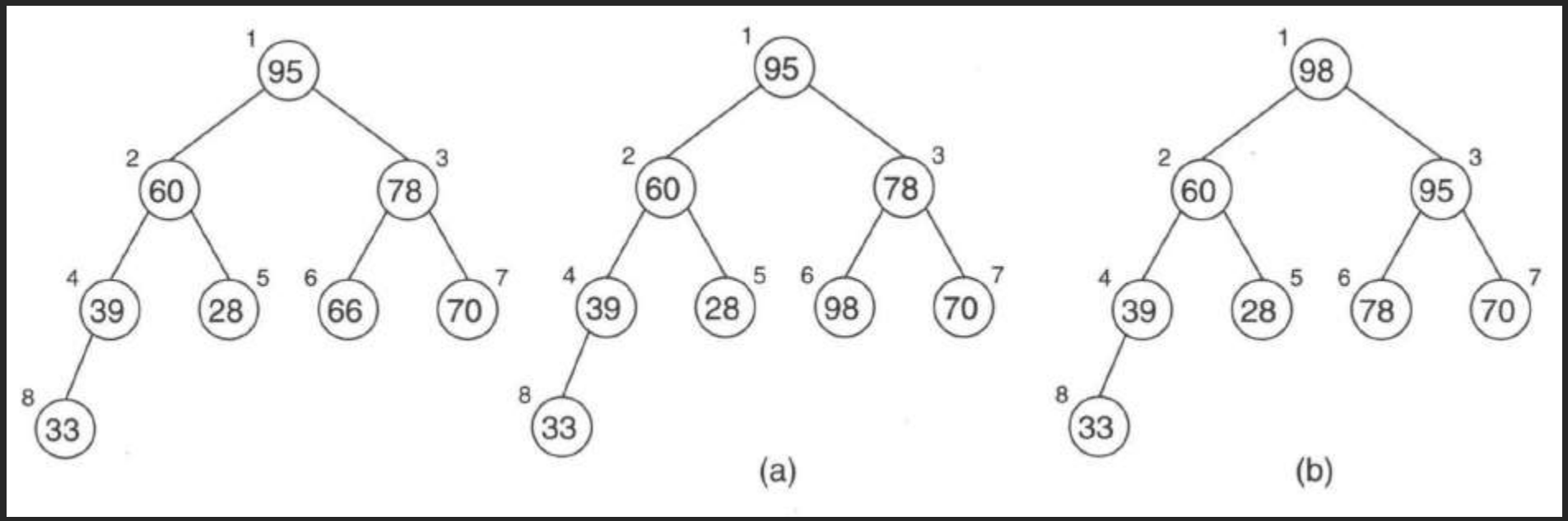
- Implementação de lista de prioridade por heap:
  - Seleção: O(1)
    - Raiz da árvore
  - Inserção: O(log n)
  - Remoção: O(log n)
  - Alteração: O(log n)
  - Construção: O(n)

Alteração de prioridades

- Alteração de prioridade de elemento: exige ajuste no heap
  - Algoritmo para realizar ajuste envolve “subir” ou “descer” na visualização por árvore
    - Subida ou descida ocorre até elemento alterado encontrar posição adequada
  - Subida: quando há aumento na prioridade
    - Trocas sucessivas com pai
  - Descida: quando há diminuição
    - Trocas sucessivas com filho de maior prioridade

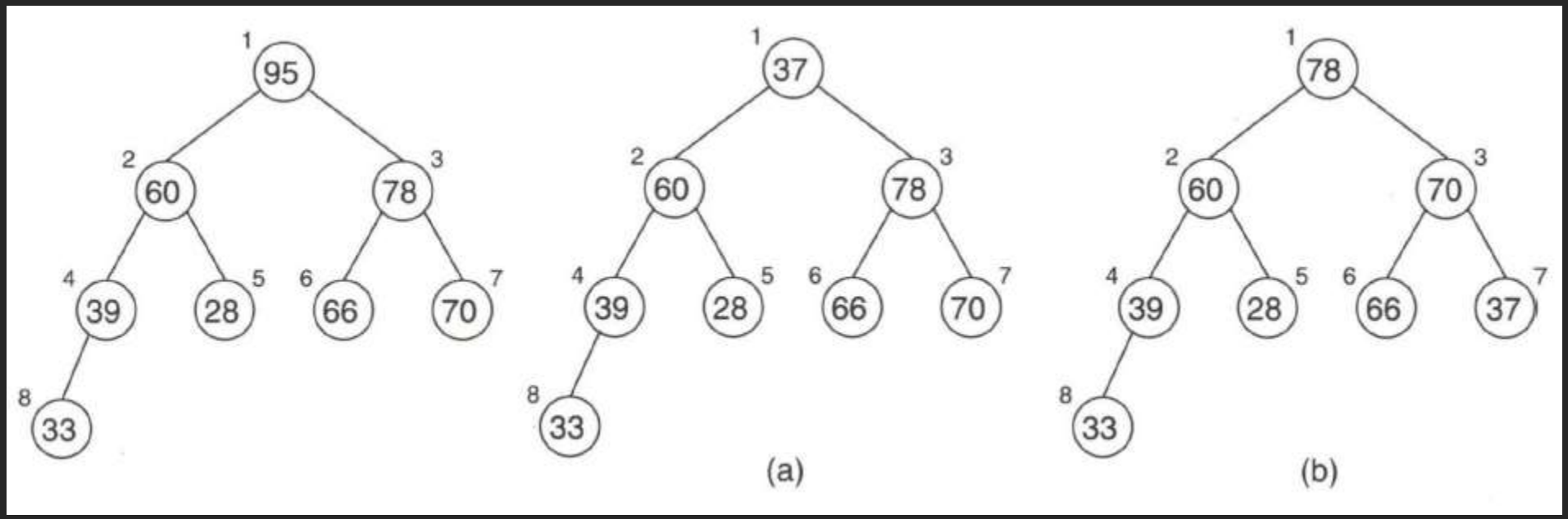
Alteração de prioridades

- Exemplo de alteração da prioridade do nó da posição 6: valor 66 para 98 (subida)



Alteração de prioridades

- Exemplo de alteração da prioridade do nó da posição 1: valor 95 para 37 (descida)



Alteração de prioridades

- Algoritmo subir: T contém a lista de prioridade

```
algoritmo 6.1: Subir por um caminho da árvore
procedimento subir(i)
    j := ⌊i/2⌋
    se j ≥ 1 então
        se T[i].chave > T[j].chave então
            T[i] ⇔ T[j]
            subir(j)
```

Alteração de prioridades

- Algoritmo descer:

```
algoritmo 6.2: Descer por um caminho da árvore
procedimento descer(i, n)
    j := 2 × i
    se j ≤ n então
        se j < n então
            se T[j + 1].chave > T[j].chave então
                j := j + 1
        se T[i].chave < T[j].chave então
            T[i] ⇔ T[j]
            descer(j, n)
```

Alteração de prioridades

- Complexidade de subir e descer: iguais
  - No pior caso: caminho da raiz até folha ou da folha até raiz
  - Altura da árvore binária completa: O(log n)
  - Complexidade: O(log n)

Inserção e remoção

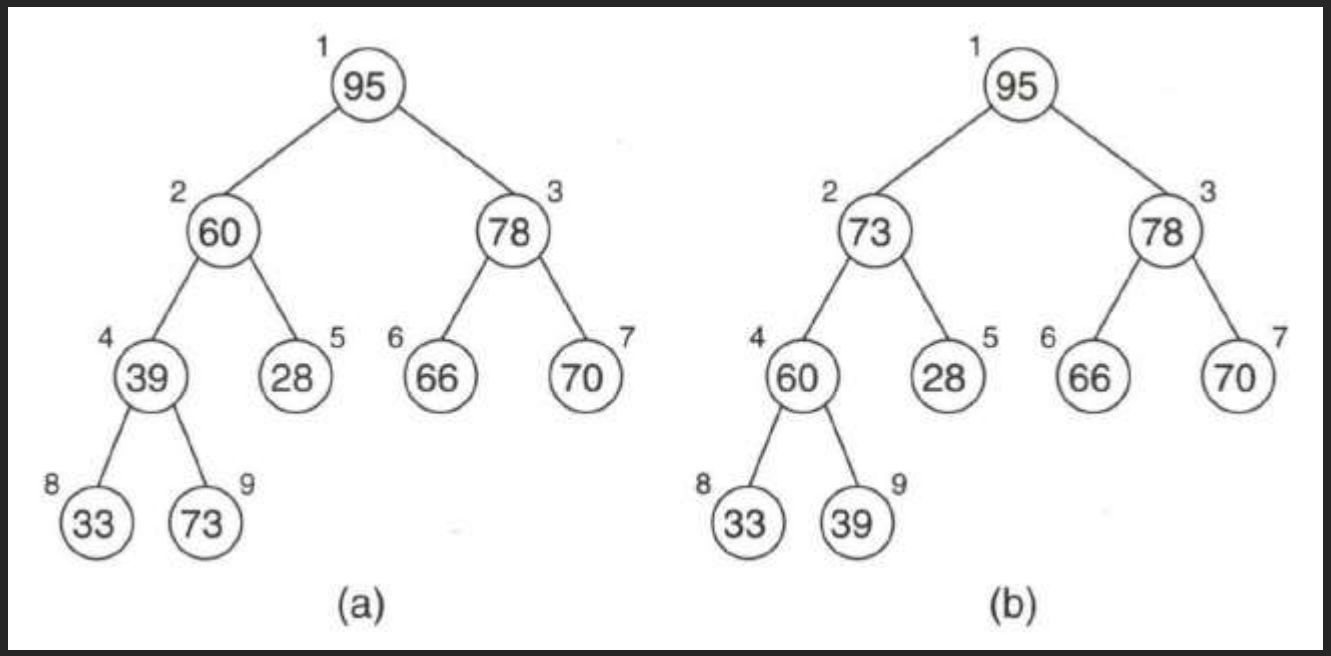
- Inserção: novo elemento é inserido como folha
  - Lista T é aumentada
  - Pode ser necessário subir elemento

```
algoritmo 6.3: Inserção em uma lista de prioridades
se n < M então
        T[n + 1] := novo
        n := n + 1
        subir(n)
senão overflow
```

  - Complexidade: O(lg n)

Inserção e remoção

- Exemplo de inserção do 73:



Inserção e remoção

- Remoção: retira o maior
  - Último elemento substitui o primeiro
  - Nova raiz possivelmente precisa descer

algoritmo 6.4: Remoção de um elemento da lista de prioridades

se $n \neq 0$ **então**

$\quad agir(\mathcal{T}[1])$

$\quad \mathcal{T}[1] := \mathcal{T}[n]$

$\quad n := n - 1$

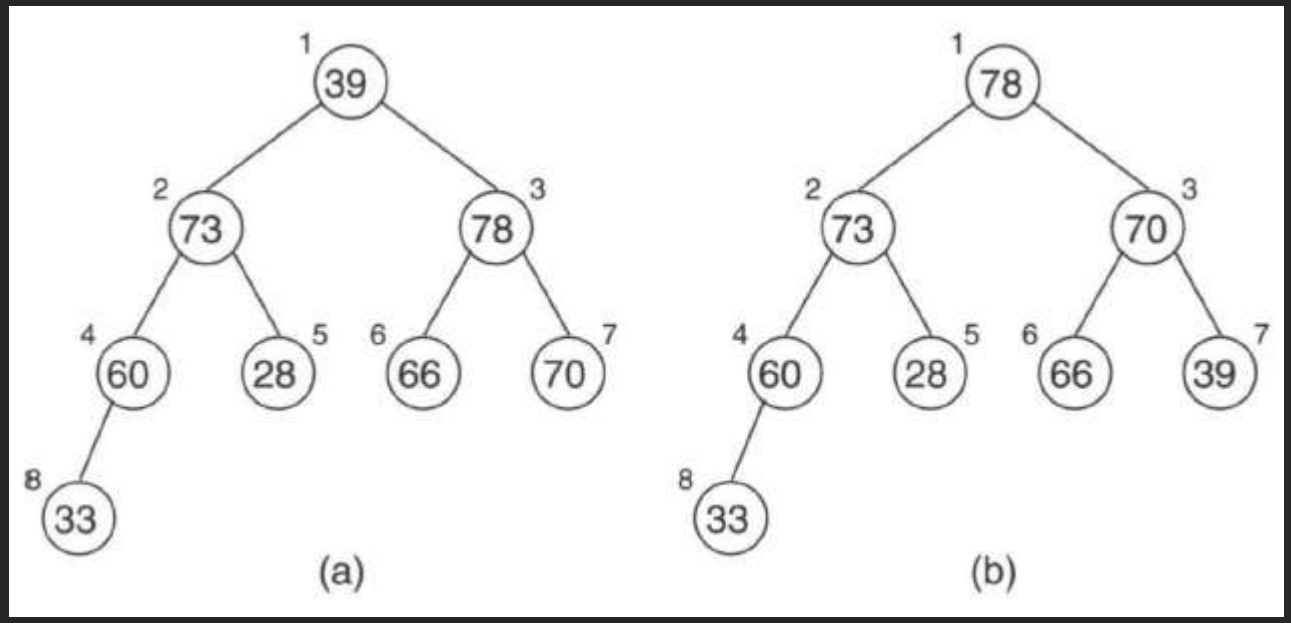
$\quad descer(1, n)$

senão $underflow$

  - Complexidade: O(lg n)

Inserção e remoção

- Exemplo de remoção:



Construção

- Construção pode ser feita através de ordenação
  - Lista ordenada (do maior para o menor) é heap
  - Custo: O(n log n)
- Construção por inserção:
  - Inicia heap com primeiro elemento
  - Insere próximos elementos um a um
  - Custo: O(n log n)

Construção

- Algoritmo de construção por inserção:

```
algoritmo 6.5: Construção da lista de prioridades
para i := 2, n faça
        subir(i)
```

Construção

- Algoritmo O(n): processo bottom-up
  - Todos elementos a partir da posição $\lfloor n/2 \rfloor + 1$ são folhas
    - Individualmente, folhas são heaps com um único nó (altura 1)
  - Heaps de altura 2 são formados juntando duas folhas ao nó pai dessas folhas + descida do pai
  - Heaps de altura 3 são formados juntando dois heaps de altura menor ao pai + descida do pai
  - ...
  - Obtém-se um heap de altura h juntando dois heaps de altura menor existentes ao pai (elemento da posição 1) + descida do pai

Construção

- Algoritmo de construção bottom-up:

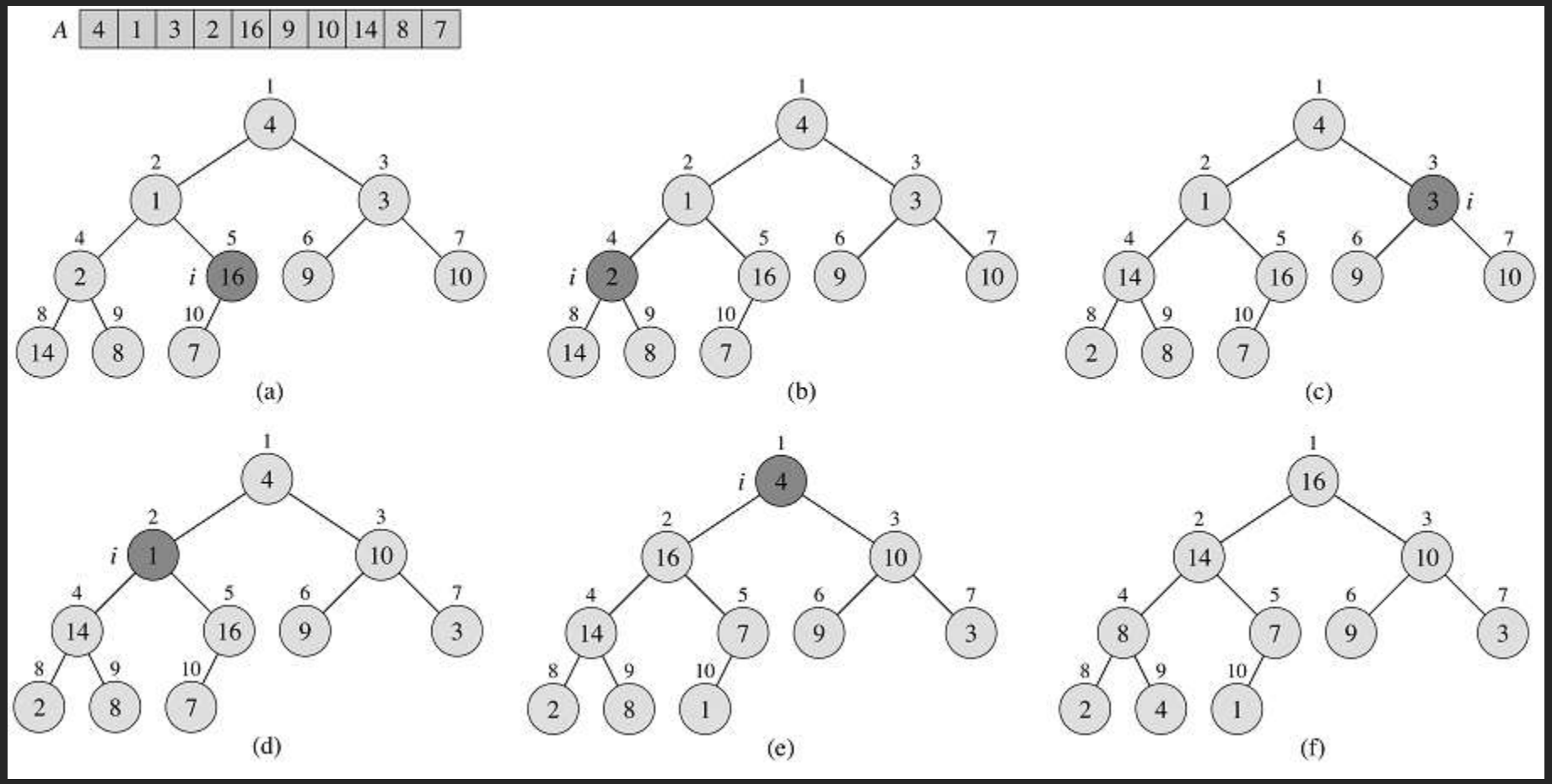
algoritmo 6.6: Construção da lista de prioridades

procedimento $arranjar(n)$

  para $i = \lfloor n/2 \rfloor, 1$ **faça**

    $descer(i, n)$

Construção



Construção

- Complexidade de tempo da construção bottom-up:
  - Há no máximo $\lceil n/2^j \rceil$ nós de altura j
  - Tempo total é aproximadamente

$$\sum_{j=2}^{h} \left[\mathrm{j} \frac{n}{2^j} \right] = n \sum_{j=2}^{h} \left[\frac{j}{2^j} \right] \leq n \sum_{j=2}^{\infty} \left[\frac{j}{2^j} \right] = n \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2} = 2n$$

o que é O(n)

Ordenação

- Algoritmo de ordenação HeapSort:
  - Constrói heap
  - Iterativamente remove maior elemento e insere no final da lista ordenada
- Complexidade:
  - Construção: O(n)
  - Remoções: O(n lg n)

Ordenação

- Algoritmo HeapSort:

algoritmo 6.7: Ordenação de $\mathcal{T}$

$arranjar(n)$

$m := n$

enquanto $m > 1$ **faça**

$\quad \mathcal{T}[1] \Leftrightarrow \mathcal{T}[m]$

$\quad m := m - 1$

$\quad descer(1, m)$

Exercícios

- 6.1, 6.6, 6.7, 6.8, 6.9 e 6.10